Desarrollo de Aplicaciones en Internet

CORREOS CORPORATIVOS



Proyecto: PROYECTO FINAL

Estudiante: Josue Jairo Quichca Alvarez

Docente: Michael Montgomery Rosell

Tecnologías principales:

Laravel · Livewire · Filament · Eloquent · Tailwind CSS · Vite

Repositorio:

https://github.com/josueqalvarez/correos_corporativos.git

2026

ÍNDICE

1. Administración de blogs (Filament)
2. Livewire para el Frontend
3. Tailwind CSS para el diseño
4. Eloquent ORM para la gestión de datos
5. Rutas
6. Arquitectura y organización del proyecto
7. Autenticación y gestión de usuarios
8. Migraciones y base de datos
9. Vite y gestión de recursos del frontend
10. Temática del proyecto: Correos corporativos

1. Administración de blogs (Filament)

El proyecto utiliza **Filament 5** como herramienta para implementar el panel administrativo encargado de gestionar el contenido del blog. La administración de los blogs se organiza mediante un **Resource**, específicamente BlogResource`, ubicado en:

```
app/Filament/Resources/Blogs/BlogResource.php
```

El BlogResource representa el recurso administrativo correspondiente a los blogs y sirve como punto central para configurar las operaciones de gestión de este contenido.

La implementación de Filament para los blogs está organizada de la siguiente manera:

```
app/
└── Filament/
    └── Resources/
        └── Blogs/
            ├── BlogResource.php
            ├── Pages/
            │   ├── CreateBlog.php
            │   ├── EditBlog.php
            │   └── ListBlogs.php
            ├── Schemas/
            │   └── BlogForm.php
            └── Tables/
                └── BlogsTable.php
```

Esta organización permite separar las diferentes responsabilidades del panel administrativo.

- **BlogResource.php**: define el recurso administrativo de los blogs.
- **CreateBlog.php**: corresponde a la página para crear nuevos blogs.
- **EditBlog.php**: corresponde a la página para modificar blogs existentes.
- **ListBlogs.php**: corresponde a la página donde se listan los blogs.
- **BlogForm.php**: contiene la configuración del formulario utilizado para gestionar los datos de un blog.
- **BlogsTable.php**: contiene la configuración de la tabla utilizada para mostrar los registros.

La implementación permite gestionar los blogs mediante las operaciones básicas de un **CRUD**. Estas operaciones se distribuyen principalmente entre las páginas y componentes definidos dentro de BlogResource`.

Ventajas

El uso de Filament aporta principalmente:

- **Desarrollo más rápido** del panel administrativo.
- **Organización mediante Resources**.

- **Formularios y tablas configurables**.
- Integración con Laravel y sus modelos.
- Reducción del código necesario para construir manualmente una interfaz administrativa.
- Separación de las funcionalidades de administración respecto a la interfaz pública del sitio.

En este proyecto, su utilización está enfocada específicamente en la **gestión administrativa de los blogs**, permitiendo crear, consultar, modificar y administrar el contenido desde un panel centralizado.

Integración con Laravel

Filament forma parte de Laravel y, en este proyecto, se integra con la estructura habitual de la aplicación.

El flujo simplificado de administración puede representarse de la siguiente manera:

```
Administrador
      │
      ▼
Panel de Filament
      │
      ▼
BlogResource
      │
      ├── BlogForm
      │
      └── BlogsTable
      │
      ▼
Modelo / Eloquent
      │
      ▼
Base de datos
```

2. Livewire para el Frontend

El proyecto utiliza **Livewire 4** para desarrollar componentes dinámicos e interactivos dentro de la aplicación. Livewire forma parte de las dependencias principales del proyecto mediante el paquete livewire/livewire`.

Esta tecnolo[gia] permite construir interfaces interactivas utilizando principalmente **PHP y Laravel**, evitando tener que implementar manualmente toda la lógica de interacción mediante JavaScript. En el proyecto se utiliza en diferentes funcionalidades de la aplicación, incluyendo páginas relacionadas con el blog, comentarios, contacto, perfil y configuración de usuarios.

El uso de Livewire permite:

- Desarrollar componentes interactivos utilizando PHP y Laravel.
- Separar la lógica de diferentes funcionalidades en componentes independientes.
- Reutilizar componentes en diferentes partes de la aplicación.
- Reducir la cantidad de JavaScript escrito manualmente.

- Integrar la lógica de la interfaz directamente con Laravel.

Livewire actúa como una capa de interacción entre la lógica de Laravel y las vistas que utiliza el usuario.

Estructura de los componentes Livewire

Los componentes Livewire se encuentran principalmente dentro de:

```
Livewire/
├── Actions/
│   └── Logout.php                    // Cierre de sesión del usuario
│
├── Pages/
│   ├── Blog/
│   │   ├── BlogHome.php              // Página principal del blog
│   │   └── BlogArticulo.php          // Visualización de un artículo del
blog
│   │
│   ├── Users/
│   │   └── CommentSection.php        // Gestión y visualización de
comentarios
│   │
│   ├── Contactanos.php               // Página de contacto
│   ├── DominioYHostingEmail.php      // Información sobre dominio y
hosting de correo
│   ├── Faq.php                       // Página de preguntas frecuentes
│   ├── Home.php                      // Página principal
│   ├── Opiniones.php                 // Página relacionada con opiniones
de clientes
│   └── QueIncluye.php                // Información sobre lo que incluye
el servicio
│
└── Settings/
    ├── TwoFactor/                    // Funcionalidades relacionadas con
autenticación de dos factores
    ├── Appearance.php                // Configuración de apariencia
    ├── DeleteUserForm.php            // Eliminación de la cuenta del
usuario
    ├── Profile.php                   // Gestión del perfil del usuario
    └── Security.php                  // Configuración de seguridad de la
cuenta
```

Componentes del blog

Una de las principales aplicaciones de Livewire dentro del proyecto se encuentra en el sistema de blogs.

BlogHome

El corresponde a la página principal del blog, y permite gestionar la lógica asociada a la presentación de los artículos disponibles en el blog.

El flujo puede representarse de la siguiente manera:

```
Usuario
   │
   ▼
BlogHome
   │
   ▼
Obtención de artículos
   │
   ▼
Vista del blog
```

BlogArticulo

Este componente está relacionado con la visualización individual de un artículo del blog. El flujo conceptual es:

```
Usuario
   │
   ▼
Selecciona un artículo
   │
   ▼
BlogArticulo
   │
   ▼
Artículo completo
```

CommentSection

Este componente permite separar la funcionalidad relacionada con los comentarios de los artículos. De esta forma, el sistema de blog se divide en componentes con responsabilidades específicas

```
Blog
├── BlogHome
│   └── Listado de artículos
│
├── BlogArticulo
│   └── Artículo individual
│
└── CommentSection
    └── Comentarios
```

Otros componentes

El proyecto también contiene componentes con diferentes funcionalidades de la aplicación. Por ejemplo:

```
app/Livewire/Pages/
├── Home.php
├── Opiniones.php
├── QueIncluye.php
├── Contactanos.php
└── DominioYHostingEmail.php
```

También existen componentes relacionados con la gestión de usuarios y configuraciones:

```
app/Livewire/
├── Actions/
│   └── Logout.php
│
└── Settings/
    ├── Profile.php
    ├── Security.php
    ├── Appearance.php
    └── DeleteUserForm.php
```

Flujo de interacción

De manera general, la interacción de un componente Livewire puede representarse de la siguiente forma:

```
Usuario
   │
   │ Interacción
   ▼
Componente Livewire
   │
   │ Solicitud al servidor
   ▼
Laravel / PHP
   │
   │ Procesamiento
   ▼
Componente Livewire
   │
   │ Actualización
   ▼
Interfaz
```

Esto permite implementar comportamientos interactivos sin tener que desarrollar manualmente toda la comunicación entre JavaScript y el servidor.

3. Tailwind CSS para el diseño

El proyecto utiliza **Tailwind CSS 4** como framework para la construcción y personalización de los estilos de la interfaz web. También se utiliza Vite como herramienta para procesar los recursos del frontend.

La utilización de esta tecnologia permite:

- Construir interfaces rápidamente.
- Aplicar estilos directamente desde las vistas.
- Utilizar un sistema de clases reutilizables.
- Crear diseños responsive.
- Evitar una gran cantidad de CSS personalizado.
- Integrarse con Blade y Livewire.
- Mantener los estilos relacionados directamente con los elementos que modifican.

Además, al utilizar Tailwind junto con Vite, los recursos frontend pueden procesarse mediante el flujo de construcción utilizado por el proyecto.

Integración de Tailwind CSS en el proyecto

La configuración del frontend se encuentra relacionada principalmente con los siguientes archivos:

```
package.json
vite.config.js
resources/
└── css/
    └── app.css
```

El archivo package.json declara las dependencias necesarias para utilizar Tailwind CSS:

```
"dependencies": {
    "@tailwindcss/vite": "^4.1.11",
    "concurrently": "^9.0.1",
    "laravel-vite-plugin": "^3.1",
    "tailwindcss": "^4.0.7",
    "vite": "^8.0.0"
}
```

Esto permite integrar Tailwind con el proceso de construcción del frontend mediante Vite.

Uso de Tailwind con Blade

En el proyecto, Tailwind se utiliza junto con las vistas de Laravel y los componentes Livewire. El flujo puede representarse de la siguiente manera:

```
Componente Livewire
        │
        ▼
     Vista Blade
        │
        ▼
Clases de Tailwind CSS
        │
        ▼
Interfaz visual
```

Organización de los estilos

El proyecto contiene el archivo:

```
resources/css/app.css
```

Este archivo forma parte de los recursos CSS procesados por el sistema de frontend. El repositorio también contiene la configuración necesaria para integrar Tailwind con Vite.

La estructura general del frontend puede representarse así:

```
resources/
├── css/
│   └── app.css
│
└── views/
    └── ...
```

Las vistas utilizan las clases de Tailwind para definir la presentación de los diferentes elementos de la aplicación.

Integración con Vite

Tailwind CSS está integrado con **Vite**, que se utiliza para procesar y construir los recursos del frontend.

El proyecto utiliza los siguientes scripts:

```
"scripts": {
    "build": "vite build",
    "dev": "vite"
}
```

Durante el desarrollo se puede utilizar:

```
npm run dev
```

Por lo tanto, la relación entre estas herramientas puede representarse de la siguiente manera:

```
Laravel
   │
   ▼
Vite
   │
   ├── Tailwind CSS
   │
   └── Recursos frontend
           │
           ▼
       Navegador
```

Relación con Livewire y Laravel

La relación entre las tecnologías puede resumirse así:

```
                Laravel
                   │
                   ▼
               Livewire
                   │
                   ▼
                Blade
                   │
                   ▼
            Tailwind CSS
                   │
                   ▼
           Interfaz visual
```

Mientras Laravel proporciona la estructura principal de la aplicación y Livewire permite implementar componentes dinámicos, Tailwind CSS se encarga de definir la apariencia de dichos componentes. Por esta razón, Tailwind no reemplaza a Livewire ni a Laravel, sino que complementa estas tecnologías desde la parte visual del proyecto.

Archivos y tecnologías relacionadas

Los principales elementos relacionados con Tailwind CSS son:

```
package.json
    │
```

```
    ├── tailwindcss
    ├── @tailwindcss/vite
    ├── vite
    └── laravel-vite-plugin

resources/
└── css/
    └── app.css

vite.config.js
```

El package.json confirma la utilización de Tailwind CSS 4, Vite y el plugin de integración con Laravel. En conjunto, estos elementos permiten integrar Tailwind CSS dentro del proceso de desarrollo frontend de la aplicación.

4. Eloquent ORM para la gestión de datos

El proyecto utiliza **Eloquent ORM**, el sistema de mapeo objeto-relacional incluido en Laravel, para trabajar con los datos de la aplicación. Su utilización permite establecer una conexión entre la lógica de la aplicación y la información almacenada en la base de datos.

La utilización de Eloquent en el proyecto proporciona varias ventajas:

- Permite trabajar con la base de datos mediante modelos PHP.
- Facilita la construcción de consultas.
- Permite definir relaciones entre entidades.
- Proporciona métodos para crear, consultar, actualizar y eliminar registros.
- Reduce la necesidad de escribir consultas SQL directamente.
- Integra la gestión de datos con el resto del framework Laravel.
- Facilita la reutilización de la lógica relacionada con las entidades.

Esta tecnologia constituye una parte importante del proyecto porque permite conectar la lógica desarrollada en Laravel con la información almacenada en la base de datos.

Su integración con **Filament** permite administrar los registros desde el panel administrativo, mientras que su integración con **Livewire** permite utilizar esos mismos datos en las interfaces públicas de la aplicación.

Por lo tanto, Eloquent actúa como una capa intermedia entre la aplicación Laravel y la base de datos, proporcionando una forma estructurada y orientada a objetos para trabajar con la información del sistema.

Eloquent y Filament

Eloquent tiene una relación importante con **Filament**, utilizado en el proyecto para administrar los blogs.

El flujo puede representarse de la siguiente manera:

```
Administrador
      │
      ▼
```

```
Panel de Filament
      |
      ▼
BlogResource
      |
      ▼
Modelo Eloquent
      |
      ▼
Base de datos
```

Filament proporciona la interfaz administrativa, mientras que Eloquent se encarga de representar y gestionar los datos que se encuentran en la base de datos.

Por esta razón, ambos componentes cumplen funciones diferentes:

Tecnología	Responsabilidad
Filament	Interfaz administrativa
Eloquent	Gestión de los datos
Base de datos	Almacenamiento de los datos

Eloquent y Livewire

Eloquent también puede trabajar junto con los componentes Livewire utilizados en el proyecto.

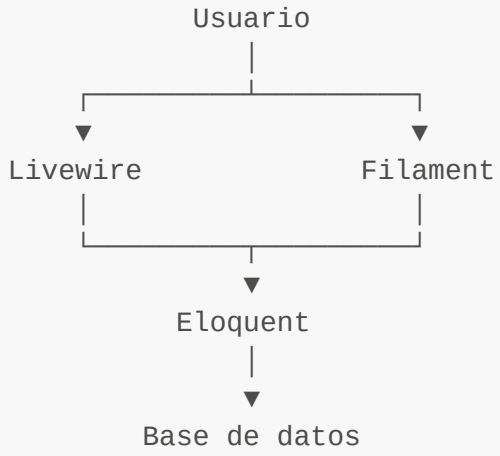
El flujo puede representarse como:

```
Usuario
   |
   ▼
Componente Livewire
   |
   ▼
Eloquent
   |
   ▼
Base de datos
   |
   ▼
Eloquent
   |
   ▼
Livewire
   |
   ▼
Vista Blade
```

Tal es el caso cuando vemos que un componente relacionado con el blog puede consultar los artículos utilizando un modelo Eloquent y posteriormente enviar esos datos a la vista para mostrarlos al usuario.

Relación con la arquitectura del proyecto

Eloquent forma parte de una cadena de componentes dentro de la aplicación:



En esta arquitectura, diferentes partes de la aplicación pueden utilizar los mismos modelos Eloquent para acceder a la información almacenada.

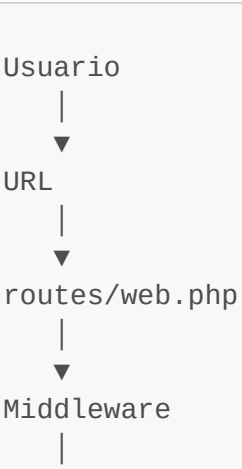
Esto permite mantener una separación de responsabilidades:

- **Livewire:** interacción y componentes dinámicos.
- **Filament:** administración de información.
- **Eloquent:** interacción con los datos.

5. Rutas

El proyecto utiliza el sistema de rutas de **Laravel** para definir las direcciones URL disponibles en la aplicación y determinar qué componente debe ejecutarse cuando un usuario realiza una petición. Las rutas principales de la aplicación web se encuentran en el archivo routes/web.php.

El flujo general puede representarse de la siguiente manera:



```
   ▼
Componente Livewire / Controlador
   │
   ▼
Respuesta
```

Breve resumen de las rutas existentes:

Ruta (URL)	Componente Livewire	Nombre de ruta	Funcionamiento
/	pages.home	home	Muestra la página principal del sitio.
/blog	pages.blog.blog-home	blog	Muestra el listado de artículos del blog.
/blog/{slug}	pages.blog.blog-articulo	blog-articulo	Muestra un artículo específico utilizando su slug.
/que-incluye	pages.que-incluye	que-incluye	Muestra la información sobre lo que incluye el servicio.
/dominio-y-hosting-email	pages.dominio-y-hosting-email	dominio-y-hosting-email	Presenta información relacionada con dominio y hosting de correo.
/opiniones	pages.opiniones	opiniones	Ruta destinada a opiniones, actualmente **desactivada** porque está comentada.
/faq	pages.faq	faq	Muestra las preguntas frecuentes del servicio.
/contacto	pages.contactanos	contacto	Permite acceder a la sección de contacto.
/login	pages.users.auth.login	login	Página de inicio de sesión.
/crear-cuenta	pages.users.auth.create	crear-cuenta	Página para crear una nueva cuenta.
/profile	pages.users.profile	profile	Página correspondiente al perfil del usuario.

Estructura de las rutas

```
routes/web.php
│
├── Página principal
│   └── /
│
├── Blog
│   ├── /blog
│   └── /blog/{slug}
```

```
│
├── Información del servicio
│   ├── /que-incluye
│   └── /dominio-y-hosting-email
│
├── Información adicional
│   ├── /faq
│   └── /opiniones [desactivada]
│
├── Contacto
│   └── /contacto
│
└── Usuarios
    ├── /login
    ├── /crear-cuenta
    └── /profile
```

6. Arquitectura y organización del proyecto

La organización del proyecto facilita el desarrollo porque cada parte tiene una responsabilidad determinada. Por ejemplo, si se necesita modificar una consulta relacionada con una entidad, se puede buscar el modelo correspondiente dentro de app/Models/. Si se necesita modificar una página dinámica, se puede trabajar sobre su componente dentro de app/Livewire/. De manera similar, la administración de los blogs se encuentra organizada dentro de app/Filament/Resources/Blogs/`.

En conjunto, la estructura utilizada permite organizar el proyecto siguiendo las convenciones de Laravel y separar las responsabilidades entre **lógica de aplicación, componentes dinámicos, administración, vistas, estilos, rutas y base de datos**.

```
correos_corporativos/
│
├── app/
│   ├── Filament/
│   ├── Livewire/
│   └── Models/
│
├── database/
│   ├── factories/
│   ├── migrations/
│   └── seeders/
│
├── resources/
│   ├── css/
│   └── views/
│
├── routes/
│   └── web.php
│
├── composer.json
```

```
├── package.json
└── vite.config.js
```

app/

El directorio app/ contiene gran parte de la lógica principal de la aplicación Laravel.

app/Models/ Contiene los **modelos Eloquent** utilizados para representar las entidades de la aplicación.

app/Livewire/ Contiene los componentes **Livewire** utilizados para implementar las diferentes funcionalidades y páginas dinámicas de la aplicación.

```
app/
└── Livewire/
    ├── Pages/
    ├── Actions/
    └── Settings/
```

Dentro de Pages se encuentran componentes relacionados con diferentes páginas del proyecto, incluyendo los componentes del blog.

```
Pages/
└── Blog/
    ├── BlogHome.php
    ├── BlogArticulo.php
    └── CommentSection.php
```

app/Filament/ Contiene los recursos utilizados para el **panel administrativo de Filament**.

```
app/
└── Filament/
    └── Resources/
        └── Blogs/
```

En el caso del blog, esta estructura contiene el BlogResource y sus diferentes páginas, formularios y tablas.

database/

Contiene elementos relacionados con la base de datos. Su estructura principal es:

```
database/
├── factories/
├── migrations/
└── seeders/
```

migrations/. Contiene las migraciones, utilizadas para definir y modificar la estructura de las tablas de la base de datos. Permiten mantener la estructura de la base de datos mediante archivos versionados dentro del proyecto.

Factories y seeder se utilzan en conjunto para la creacion de datos aleatorios para poder implementar en pruebas, pero no fueron utilizados.

resources/

El directorio resources/ contiene recursos utilizados por la interfaz de la aplicación. En el proyecto se encuentran principalmente:

```
resources/
├── css/
└── views/
```

resources/views/ Contiene las vistas **Blade** utilizadas para representar la interfaz.

resources/css/. Contiene los recursos CSS utilizados por el frontend. Dentro de esta estructura se encuentra app.css, que forma parte del flujo de estilos de la aplicación y se integra con Tailwind CSS.

routes/. El directorio routes/ contiene los archivos encargados de definir las rutas de la aplicación.

Archivos de configuración principales

composer.json. Este archivo define las dependencias relacionadas con PHP y Laravel. Permite definir las dependencias que necesita el backend del proyecto.

package.json Este archivo contiene las dependencias relacionadas con el frontend. También define comandos utilizados durante el desarrollo

vite.config.js. Este archivo contiene la configuración de **Vite**, utilizado para procesar y construir los recursos frontend.

Responsabilidades

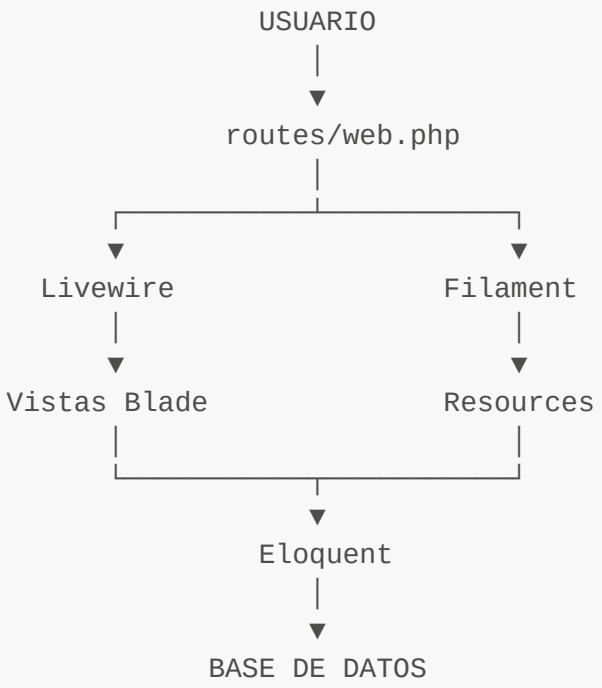
La estructura del proyecto permite dividir las responsabilidades entre diferentes componentes:

Parte	Responsabilidad
Models/	Representar y gestionar datos mediante Eloquent
Livewire/	Componentes dinámicos e interacción

Parte	Responsabilidad
Filament/	Administración de contenido
routes/	Definición de rutas
resources/views/	Vistas Blade
resources/css/	Recursos de estilos
database/migrations/	Estructura de la base de datos
database/factories/	Generación de datos de prueba
database/seeders/	Inserción de datos iniciales
composer.json	Dependencias PHP
package.json	Dependencias frontend
vite.config.js	Configuración de Vite

Relación entre los componentes

La arquitectura general del proyecto puede resumirse mediante el siguiente flujo:



7. Autenticación y gestión de usuarios

El proyecto implementa funcionalidades relacionadas con la **autenticación y gestión de usuarios**. Estas funcionalidades permiten controlar el acceso de los usuarios a determinadas partes de la aplicación, administrar sus cuentas y proporcionar opciones relacionadas con el perfil y la seguridad.

Componentes relacionados con usuarios

La gestión de usuarios está organizada mediante diferentes componentes Livewire.

Entre los componentes presentes en el proyecto se encuentran:

```
app/
└── Livewire/
    ├── Actions/
    │   └── Logout.php
    │
    └── Settings/
        ├── Profile.php
        ├── Security.php
        ├── Appearance.php
        └── DeleteUserForm.php
```

app/Livewire/Actions/Logout.php. Este componente está relacionado con la acción de **cerrar sesión** del usuario.

app/Livewire/Settings/Profile.php. Este componente permite centralizar funcionalidades relacionadas con la información del perfil del usuario. La utilización de Livewire permite que estas funcionalidades puedan implementarse como componentes independientes dentro de la aplicación.

app/Livewire/Settings/Security.php. Este componente está relacionado con las opciones de seguridad disponibles para la cuenta del usuario.

app/Livewire/Settings/Appearance.php. Este componente está relacionado con la configuración de la apariencia de la aplicación. Esta funcionalidad forma parte de la sección de configuración del usuario.

app/Livewire/Settings/DeleteUserForm.php. Este componente está destinado a la funcionalidad relacionada con la eliminación de la cuenta del usuario.

Protección mediante middleware

La autenticación se puede complementar con **middleware**, que permite comprobar determinadas condiciones antes de permitir el acceso a una ruta.

Un ejemplo habitual es el middleware auth`.

El flujo puede representarse de la siguiente manera:

```
Usuario
   │
   ▼
Solicita una ruta protegida
   │
   ▼
Middleware de autenticación
   │
```

```
    ├── Autenticado ──────▶ Acceso
    │
    └── No autenticado ──▶ Redirección / rechazo
```

Esto permite diferenciar entre las partes públicas y privadas de una aplicación.

8. Migraciones y base de datos

El proyecto utiliza el sistema de migraciones de Laravel para definir y gestionar la estructura de la base de datos. Las migraciones permiten describir mediante código las tablas, columnas, claves y otras modificaciones que deben aplicarse a la base de datos. Esta característica es importante porque permite que la estructura de la base de datos forme parte del código fuente del proyecto, en lugar de depender únicamente de una base de datos creada manualmente.

Ubicación de las migraciones

Las migraciones de Laravel se almacenan dentro del directorio:

```
database/
└── migrations/
```

Esta ubicación forma parte de la estructura estándar de Laravel y permite mantener agrupados los archivos responsables de definir la estructura de la base de datos.

Migraciones y Eloquent

Las migraciones y Eloquent cumplen funciones diferentes, pero trabajan conjuntamente.

```
Migration
    │
    ▼
Define estructura
    │
    ▼
Base de datos
    ▲
    │
Eloquent
    │
    ▼
Modelos
```

La migración se encarga principalmente de **definir la estructura de la base de datos**, mientras que Eloquent permite **trabajar con los datos almacenados en esas tablas**.

Por ejemplo:

```
Migration
    ↓
Tabla blogs
    ↓
Modelo Blog
    ↓
Eloquent
    ↓
Aplicación
```

Migraciones y Filament

Las migraciones también tienen relación con el panel administrativo de Filament.

En el caso del sistema de blogs:

```
Migration
    │
    ▼
Tabla de blogs
    │
    ▼
Modelo Eloquent
    │
    ▼
BlogResource
    │
    ▼
Panel de Filament
```

La migración define la estructura que necesita la base de datos, Eloquent permite trabajar con los registros y Filament proporciona la interfaz administrativa para gestionarlos.

9. Vite y gestión de recursos del frontend

El proyecto utiliza Vite como herramienta para gestionar y construir los recursos del frontend. Vite se integra con Laravel y permite procesar recursos como archivos CSS y JavaScript utilizados por la aplicación.

En el proyecto, Vite trabaja junto con Tailwind CSS, permitiendo procesar los estilos utilizados en las vistas y generar los recursos que posteriormente serán utilizados por el navegador.

La relación general puede representarse de la siguiente manera:

```
Laravel
   │
   ▼
```

```
Vite
   │
   ├── CSS
   ├── JavaScript
   └── Tailwind CSS
          │
          ▼
      Recursos frontend
          │
          ▼
        Navegador
```

El uso de Vite en el proyecto proporciona varias ventajas:

- Permite gestionar los recursos frontend de forma organizada.
- Facilita el desarrollo de CSS y JavaScript.
- Se integra con Laravel.
- Permite trabajar conjuntamente con Tailwind CSS.
- Proporciona un entorno de desarrollo para trabajar con los recursos del frontend.
- Permite generar una construcción optimizada para producción.
- Centraliza parte del proceso de construcción del frontend mediante package.json y vite.config.js`.

Configuración mediante package.json`

El archivo **package.json** define las dependencias y scripts relacionados con el frontend. Entre las dependencias utilizadas se encuentran:

```
tailwindcss
@tailwindcss/vite
vite
laravel-vite-plugin
```

Estas herramientas permiten integrar Vite con Laravel y Tailwind CSS.

La relación puede representarse así:

```
package.json
     │
     ├── Vite
     ├── Tailwind CSS
     └── Laravel Vite Plugin
            │
            ▼
      Frontend del proyecto
```

Integración con Tailwind CSS

Una de las principales funciones de Vite dentro del proyecto es trabajar conjuntamente con Tailwind CSS.

La relación entre ambas tecnologías puede representarse como:

```
Vista Blade
     |
     ▼
Clases Tailwind
     |
     ▼
Tailwind CSS
     |
     ▼
Vite
     |
     ▼
CSS procesado
     |
     ▼
Navegador
```

Tailwind proporciona los estilos correspondientes, mientras que Vite participa en el proceso de construcción de los recursos utilizados por la aplicación.

Archivo vite.config.js`

El proyecto también contiene **vite.config.js**. Este archivo se utiliza para configurar Vite y establecer cómo debe trabajar con el proyecto Laravel.

Conceptualmente:

```
vite.config.js
       |
       ▼
Configuración de Vite
       |
       ▼
Procesamiento de recursos
```

La configuración permite conectar Vite con el ecosistema de Laravel y con los recursos frontend utilizados por la aplicación.

Vite durante el desarrollo

Durante el desarrollo, el flujo puede representarse así:

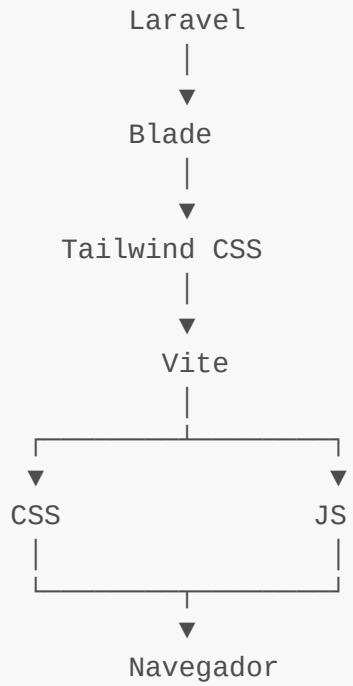
```
Desarrollador
      |
```

```
      ▼
Modifica código
      │
      ▼
Recursos CSS / JS
      │
      ▼
Vite
      │
      ▼
Aplicación
      │
      ▼
Navegador
```

Esto permite trabajar de manera más rápida, ya que los cambios realizados en los recursos frontend pueden ser procesados durante el desarrollo.

Relación con las demás tecnologías

Vite forma parte de la capa frontend del proyecto y trabaja conjuntamente con otras tecnologías:



El proyecto **Correos Corporativos** es una aplicación web orientada a presentar y promocionar servicios relacionados con la implementación de **correos electrónicos corporativos para empresas**. La aplicación utiliza Laravel como framework principal y combina diferentes tecnologías para ofrecer información sobre el servicio, gestionar contenido mediante un blog y proporcionar funcionalidades de interacción con los usuarios.

La temática principal se centra en explicar la importancia de utilizar direcciones de correo asociadas al dominio de una empresa, en lugar de depender exclusivamente de cuentas de correo personales. Por

ejemplo, una empresa puede utilizar direcciones como ventas@empresa.com, `contacto@empresa.com` o `gerencia@empresa.com`.